An insight into tree based machine learning techniques for big data Analytics using Apache Spark

Ananthi Sheshasaayee
Associate Professor, Research Supervisor
Head Department of Computer Science
Quaid – E – Millath Govt College for Women
Chennai, India
Ananthi.research@gmail.com

J V N Lakshmi
Assistant Professor[1], Research Scholar[2]
[1]IT Department, AIMS Institutions, Bangalore.
[2]SCSVMV University, Kanchipuram.
India.
jlakshmi.research@gmail.com

Abstract— **Data Analysis, Classification and Regression on Big Data using machine learning algorithms in a novel approach used in various science and medical streams. Map Reduce is widely used frame work to parallelize machine learning algorithms. These algorithms are tuned to attain best outcomes. This techniques uses lots of time for processing map reduce model multiple times by tuning the parameters as per the requirement. To achieve the shortest time consumption for tuning the jobs, Apache Spark based model is proposed for optimizing the assignments. This model is to predict the temperature from existing data by training using tree based machine learning techniques. This model replaces the map reduce by spark for implementing the best prediction result. The prediction outcomes computed are compared on tree structured ML methods with respect to time and space utilization.**

Keywords— Data Analysis; Big Data; Machine Learning; Decision Tree; Random Forest; Gradient Boosted tree; Apache Spark; Prediction;

I. Introduction

Big Data is lucid, enabling organisations to gain insights from new sources of data that were ever mined in past. Processing data which varies in volume, variety and veracity is beyond the capacity of typical database software to incarcerate, pile, control and evaluate.

Recently the increasing needs to gather massive amounts of data and aptitude to analyze the data in today's scientific and business world is competitive. Compute, transform, compress, extract and pile are prominent challenges which convey the drawbacks in the conventional architecture and techniques. MapReduce is an efficient parallel programming model for the data intensive applications for processing big data sets. Owing to few shortcomings in MapReduce new model are highly scalable. MapReduce uses a Hadoop distributed file system for parallel processing.

MapReduce model such as high disk reads and writes, low through put results in low performance while handling clusters. Meanwhile the express development in technology emerged platforms like Kafka, Spark, Flume and many more. Because of the drawbacks in MapReduce upcoming technologies replaced it. The recent popularity of these parallel programming models has invoked significant interest in implementing scalable versions of Machine Learning algorithms.

Machine learning techniques learn to understand complex data sets to make critical decisions and tune the features for abstracting high performance. The performance expected depends intimately on the model features underlying the parallel programming framework. This article discusses the implementation of machine learning techniques on Apache Spark framework for scaling space and execution time.

Machine learning broadly classified as supervised and unsupervised learning. Supervised learning techniques emphasis on accurate predictions and where as unsupervised learning aspire on compact descriptions of the data. Data classification, complex pattern recognition, predictions/intelligent decisions and clustering are features of machine learning techniques.

A. Existing System

The MapReduce use machine learning techniques for a single predictor. It demonstrates parallel map and reduce running the jobs carrying iterative algorithm. While this model results in high disk rates and low throughput. This can be improved by implementing the model proposed in this paper.

B. Problem Statement

Build a scalable machine learning model that provides performance that range to very huge datasets on spark framework for prediction analysis from existing data sets. Hand tuned tree based machine learning techniques are comparable in their implementation measuring their efficiencies in terms of space and time.

The paper is organizes as follows. In section II model is depicted in which machine learning algorithms are expressed in Jupyter environment and automatically parallelized on apache spark framework. Description of tree based machine learning methods Decision tree, Random Forest and Gradient Boosted Tree in section III. In section IV discuss the role of optimization and performance evaluation by executing on spark and MapReduce model. Section V reveals research results at various stages.

Authorized licensed use limited to: VIT University. Downloaded on January 13,2026 at 16:07:22 UTC from IEEE Xplore. Restrictions apply.

II. Proposed Model

Data analysis using machine learning systems examine datasets and involuntarily capture the features while processing the trial data. The training data consists of input items and expected outcomes. The result of the function can be a uninterrupted value or predict a class label of the input object. The task of the learning machine is to forecast the value of the task for a valid input object having a small number of training data.

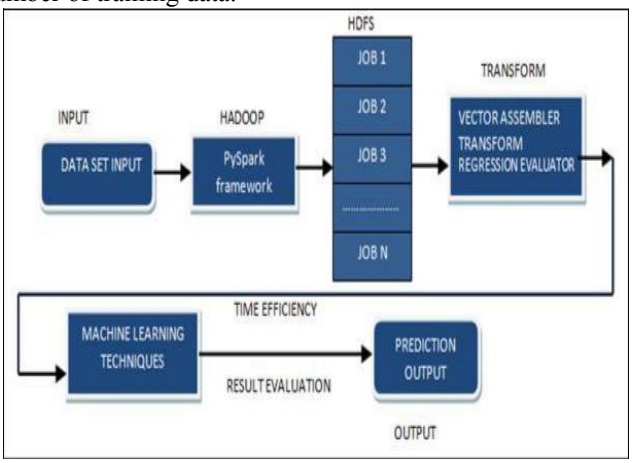

Figure 1. Framework model for prediction on datasets using ML techniques

- Read the data in spark data frame and use time method to invoke time.
- Splitting the data into train data and test data using randomSplit function from spark.
- Vector Assembler converts the data in terms of vectors.
- Transform function modifies the vectors into necessary data frames
- Mapping the labelcol and featurecol using methods from MLlib.
- Pipeline consists of stages each one operate as an estimator or a transformer when fit() is initiated.
- Regression Evaluator uses to evaluate the prediction on the featured data.
- RMSE is calculated to find the mean square error.
- Total evaluated time complexity is evaluated to process the data set.

Spark uses function pipeline to interpret the learning flow for implementing machine learning techniques. This learning flow is implemented on datasets to process the training data for attaining best test results.

III. Machine Learning Techniques

Machine Learning is one of the sub-fields of computer science emerging from artificial intelligence in the study of pattern identification and computational learning hypothesis. Forecasting from data, study of observations, learning patterns and constructing the algorithms can be explored using machine learning. These algorithms can be operated by developing a framework for trained input data set to make data driven predictions and decisions rather than following static way of programming implementing dynamically. Machine learning models incorporate statistical techniques for handling regression and classification tasks with multiple dependent and independent variables. The following methods are some of the regression techniques for predicting outcomes:

A. Decision Tree

Decision Tree is a classifier which concentrates on interpretability. This suggests a breaking down for making decisions on posing queries.

$$IG(D_P, f) - \sum_{j=1}^{m} \frac{N_j}{N_p} I(D_j) \qquad - \quad 1$$

Where IG is Gini Index IH is entropy and IE is the classification error defined for all non empty classes.

$$I_H(t) = -\sum_{i=1}^{c} p(i|t) \log_2 p(i|t) \quad - \quad 2$$

$$I_g = 1 - max\{p(i|t)\} \qquad 3$$

```
may_assembler = VectorAssembler(inputCols=["MAY"],outputCol="features")
may_assemblmn_df= may_assembler.transform(temparature_data)

(trainingData, testData) = may_assemblmn_df.randomSplit([0.8, 0.2],seed = 11)
dt = DecisionTreeRegressor(labelCol="ANNUAL", featuresCol="features")

dt.model = dt.fit(trainingData)
```

Snippet 1: Decision Tree in Jupyter Notebook

Using the decision tree in the process begins at the root and follows by splitting the data by information gain. It is an iterative process for reaching the maximum depth of the tree pruning is the process which is implemented.

B. Random Forest

A random forest is a combination of various decision tree classifiers which fits dataset as a meta-estimator for predictive accuracy and control over – fitting. This learning technique is applicable for classification and regression models. The method operates by building many decision trees as training set and it results a class of individual trees forming output. A random forest measures feature significance as the mean impurity diminishes taken from all decision trees.

Draw a random bootstrap sample of size n.

Produce a decision tree from the sample extracted by bootstrap. At every node:

1. Randomly select d features without replacement
2. Split the objective function for instance capitalizing the information gain.

Aggregate the predictions by each tree to assign the class label by majority vote.

Bias variance tradeoffs is controlled by opting large value of n and by decreasing the randomness which likely to over fit. In this implementation RandomForestClassifier the model range of the bootstrap sample is chosen to be equal to the figure of samples in the original training set by executing d = √m where m is the quantity of features in training set.

Authorized licensed use limited to: VIT University. Downloaded on January 13,2026 at 16:07:22 UTC from IEEE Xplore. Restrictions apply.

```
my_features = ['JAN', 'FEB', 'MAR', 'APR', 'MAY', 'JUN', 'JUL', 'AUG', 'SEP', 'OCT', 'NOV', 'DEC', 'JAN-FEB', 'MAR-MAY', 'JUN-SE
P', 'OCT-DEC']
temparature_data.select(my_features).show()
my_features_assembler = VectorAssembler(inputCols=my_features,outputCol="features")
my_features_df= my_features_assembler.transform(temparature_data)
(my_features_trainingData, my_features_testData) = my_features_df.randomSplit([0.8, 0.2],seed = 11)
my_features_rf = RandomForestRegressor(labelCol="ANNUAL", featuresCol="features")
my_features_predictions = my_features_rf.transform(my_features_testData)
my_features_predictions.select("prediction", "ANNUAL", "features").show(5)
my_features_testData.select(mean('ANNUAL').alias('ANNUAL value')).collect()
evaluator = RegressionEvaluator(labelCol="ANNUAL", predictionCol="prediction", metricName="rmse")
rmse = evaluator.evaluate(my_features_predictions) #
print("Root Mean Squared Error (RMSE) on test data = %g" % rmse) #0.237487
```

Snippet 2: Random Forest in Jupyter Notebook

The above snippet construct the random forest classifier from each decision trees via the n_estimator parameter and used the entropy criterion as an impurity measure to split the nodes.

C. Gradient Boosting Tree

Gradient boosting is a voracious algorithm which over fits training objects rapidly. It builds additive regression model by applying least squares method for each iteration function is parameterized to present pseudo residuals. In the present step each training data with respect to the model evaluates the pseudo - residuals which are responsible for gradient of the loss functional being minimized. For further improvement the model incorporates randomization to prove the approximation precision and execution pace of gradient boosting.

```
may_assembler = VectorAssembler(inputCols=["MAY"],outputCol="features")
may_assemblmn_df= may_assembler.transform(temparature_data)
# Spliting data into train and test
(trainingData, testData) = may_assemblmn_df.randomSplit([0.8, 0.2],seed = 11)
#DecisionTreeRegressor regression in Spark
dt = GBTRegressor(labelCol="ANNUAL", featuresCol="features")
may_model= dt.fit(trainingData)
m1 = may_model.toDebugString
m1
#Prediction on test data
may_predictions = may_model.transform(testData)
may_predictions.select("prediction", "ANNUAL", "features").show(5)
# Checking model result
evaluator = RegressionEvaluator(labelCol="ANNUAL", predictionCol="prediction", metricName="rmse")
one_feature_rmse = evaluator.evaluate(may_predictions) #
print("Root Mean Squared Error (RMSE) on test data = %g" % one_feature_rmse) #0.438126
```

Snippet 3: Gradient Boosting Tree in Jupyter Notebook

Specifically, this algorithm iterates at each draw on a subsample of the training data randomly (without replacement) from the full training data set. The model is revised for the existing iteration by arbitrarily choosing a sub sample by replacing the actual sample to fit the base learner. Gradient Boosting Tree approach increases robustness besides congestion of training sets.

IV. Performance Evaluation

In this article tree based machine learning algorithms are used for processing large temperature data set. The efficiency is observed by comparing MapReduce with Spark framework. The performance is evaluated on time and space as factors for interpreting the results. The MapReduce model shows high disk rates for read as 28.65 MB/Sec and writes operations 35.43 MB/Sec. this even reduces the performance of a processor. But executing the jobs on spark platform, the results are 17.23 MB/Sec on read and 19.41 MB/Sec for write operations.

The performance is measured in terms of time and space by examining the various tree based machine learning techniques. The observed analysis depict Gradient Boosting tree uses added space when compared with Random forest and Decision Tree. Time is another feature where Random forest and Decision tree complexity is short.

V. Expermintal Results

The machine learning techniques reveal the efficient use of time and space. These methods train the machine so they adapt to the dataset. The figure 1 shows the combined measures of learning techniques considering the time, space, read and write operations.

Table 1: Metrics on various features

Features	Map Reduce	Spark
Read	28.65	17.23
Write	35.43	19.41
Time	11.57	9.47
Space	26.54	15.43

The following graph gives the gist of the techniques used for in comparing the map reduce and spark framework on various features.

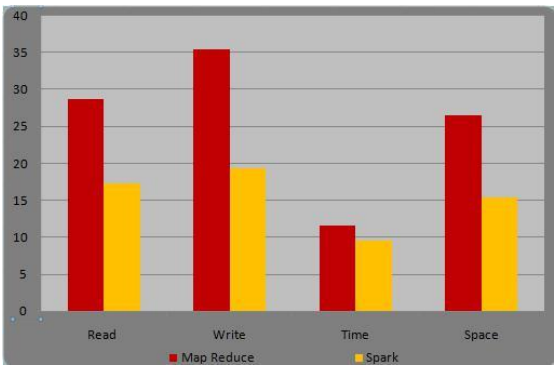

Figure 2: MapReduce and Spark performance

The following table 2 gives the tree based machine learning techniques used in performance evaluation.

Table 2: Machine Learning Algorithms

Machine Learning Techniques	Read	Write	Time	Space
Decision Tree Regressor	7.86	10.76	9.32	6.75
Random Forest Regressor	8.71	15.43	11.43	9.76
Gradient Boosting Tree Regressor	11.21	13.45	7.86	14.23

Authorized licensed use limited to: VIT University. Downloaded on January 13,2026 at 16:07:22 UTC from IEEE Xplore. Restrictions apply.

Machine learning methods are compared on time, Space, Read and write operations using Gradient Boosting Tree, Decision Tree and Random Forest techniques.

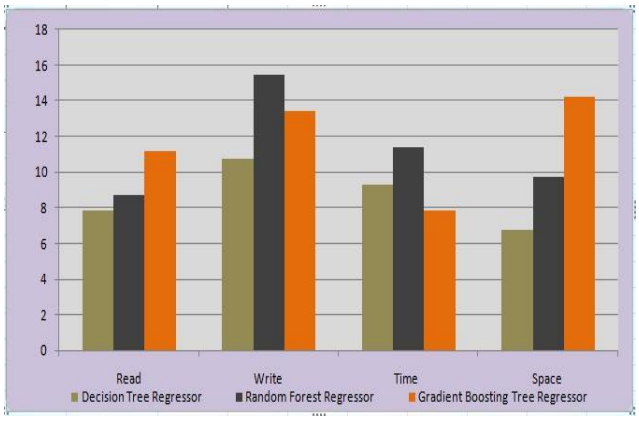

Figure 3: Machine Learning Methods

VI. Conclusion and Future work

Executing Data analysis, jobs using various parameters are observed in machine learning for efficient time and space. The proposed model optimizes the machine learning techniques on a distributed environment using spark framework to minimize the total execution time and space for future predictions. The proposed model uses Apache Spark and python as Application interface by distributing the jobs in evaluating time and space complexities for prediction.

This model for prediction can merely motivates in usage of incorporating some more additional features such as humidity, moisture, fog and pollution consistency to identify better estimate of temperature for future analysis. Much of these models are used due to the exponential growth in computing power, which allows gradually for choosing the best resulted model.

References

[1] Stuart R. and Harald B.: Beginning Python for language Research, pp. 44 – 47, (2007).

[2] Haroshi T., Shinji N. and Takuyu A.: Optimizing multiple machine learning jobs on map reduce. In IEEE – ICCCTS conference at Japan, pp. 59 – 66, (2011).

[3] C.T. Chu, Lin, Y. Yu, G. R. Bradski, A. Y. Ng, and K. Olukotun, "Map-reduce for machine learning on multicore," in NIPS, Eds. MIT Press, pp. 281–288 (2006).

[4] Jason Brownlee: Master Machine learning – How it works, pp. 1-5, (2016).

[5] Walisa and Wichan: An Adaptive ML on Map Reduce for improving performance of large scale data analysis, in EC2 IEEE, pp. 234 – 236, (2013).

[6] Asha and Sravanthi: Building Machine learning Algorithms on Hadoop for Big Data, in IJET Journal Vol 3, No 2, pp. 484 – 489, (2013).

[7] G. Schwarz: Estimating the dimension of a model, The annals of statistics, pp. 461–464, (1978).

[8] Spyder – python IDE

[9] A. Pavlo: A Comparison of Approaches to Large-Scale Data Analysis. Proc.ACM SIGMOD, (2009).

[10] Manar and Stephane : Machine Learning with Python/Scikit Learn - Application to the Estimation of Occupancy and Human Activities in SIMUREX Oct 2015.

[11] Wiley: Machine learning in Python, Predictive analysis 2015.

[12] Caruana, Rich, Nikos K, and Ainur Y. "An Empirical Evaluation of Supervised Learning in High Dimensions." Proceedings ofthe 25th International Conference on Machine Learning. ACM, 2008.

[13] Mike Bowles – Machine Learning on Big Data using Map Reduce Winter, 2012.

[14] Walisa and Wichian." An Adaptive ML on Map Reduce for improving performance of Large scale data analysis on EC2", IEEE 11th CONFERENCE on ICT and Knowledge Engineering 2013.

[15] Asha and Shravanthi, "Building Machine learning Algorithms on Hadoop for Big Data", in IJET UK Journal Vol 3, No 2 PP – 143 - 147.

[16] Apache Hive. http://hadoop.apache.org/hive.

[17] Hadoop Map/Reduce tutorial. http://hadoop.apache.org/common/docs/r0.20.0/mapred tutorial.html.

[18] Logistic regression – Wikipedia. http://en.wikipedia.org/wiki/Logistic _regression.

[19] Lakshmi J V N, " Stochastic Gradient Descent Using Linear Regression with Python" , in IJA-ERA Vol 2 Issue 8, December 2016.

[20] Lakshmi J V N, **"**Hadoop Framework for machine learning using Python**",** National conference on Advances in computer science and Engineering, Bangalore, February 2017.

Authorized licensed use limited to: VIT University. Downloaded on January 13,2026 at 16:07:22 UTC from IEEE Xplore. Restrictions apply.